

这个冷却节点的设计是一个非常经典的“蓝图异步任务 (Blueprint Async Action)”案例。

它的核心设计目标是：在蓝图里，像“监听器”一样挂在后台。当技能进入冷却时通知我（更新 UI 显示冷却时间），当技能冷却结束时通知我（隐藏 UI 或开启按钮）。

如果没有这个节点，你可能需要在 Tick 里每帧去查“还没好吗？”，这非常消耗性能。这个节点的设计思路是“事件驱动”：只有状态变了，我才通知你。

下面我将分三个部分：设计思路、头文件解析、源文件代码逐行解析 来详细讲解。

第一部分：设计思路（为什么要这么写？）

我们要实现一个节点，它有两个“出口”（执行引脚）：

1. **CooldownStart**：当冷却开始时触发，告诉我还要转多久 (TimeRemaining)。
2. **CooldownEnd**：当冷却结束时触发，告诉我完事了。

为了在 C++ 实现这个，我们需要继承 UBlueprintAsyncActionBase。

- **输入**：在这个节点开始时，我需要知道我要监听哪个 ASC（技能组件），以及监听哪个 Tag（比如 Ability.Fire.Cooldown）。
 - **过程**：
 - **监听开始**：我绑定 ASC 的 OnActiveGameplayEffectAdded。如果有新的 GE (Gameplay Effect) 加进来，且这个 GE 带着冷却 Tag，说明“冷却开始了”。
 - **监听结束**：我绑定 ASC 的 RegisterGameplayTagEvent。如果冷却 Tag 的数量变成了 0，说明“冷却结束了”。
 - **销毁**：当节点不用了，或者对象被销毁时，记得取消监听，防止崩溃。
-

第二部分：头文件 (.h) 解析

这个文件定义了“数据结构”和“接口”。

C++

```
// WaitCooldownChange.h
```

```
#pragma once
```

```
#include "CoreMinimal.h"
```

```
#include "AbilitySystemComponent.h"
```

```
#include "Kismet/BlueprintAsyncActionBase.h"
```

```
#include "WaitCooldownChange.generated.h"
```

```
// 1. 定义一个“广播员”类型
```

```
// 这是一个动态多播委托。意思是：它可以被蓝图绑定，可以传参数（float  
TimeRemaining）。
```

```
// 这里的名字 FCooldownChangeSignatuer 是类型名（注意你代码里有个拼写错误  
Signature 写成了 Signatuer，不过不影响运行）
```

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FCooldownChangeSignatuer,  
float,TimeRemaining);
```

```
/**
```

```
 * 类定义：继承自 UBlueprintAsyncActionBase
```

```
 * 这意味着这个类会自动变成蓝图里的一个“异步节点”（右上角有时钟图标的那种）
```

```
*/
```

```
UCLASS()
```

```
class AURA_API UWaitCooldownChange : public UBlueprintAsyncActionBase
```

```
{
```

```
    GENERATED_BODY()
```

```
public:
```

```
    // 2. 定义两个输出引脚
```

```
    // UPROPERTY(BlueprintAssignable) 会让这两个变量变成蓝图节点的“输出执行流”
```

```
    UPROPERTY(BlueprintAssignable)
```

```
    FCooldownChangeSignatuer CooldownStart; // 冷却开始引脚
```

```
UPROPERTY(BlueprintAssignable)
```

```
FCooldownChangeSignatuer CooldownEnd;    // 冷却结束引脚
```

```
// 3. 工厂函数 (最重要!)
```

```
// 这个函数会创建这个节点的实例。
```

```
// meta=(BlueprintInternalUseOnly = "true") 表示这个函数是给蓝图引擎内部用的,
```

```
// 它负责生成节点, 用户在蓝图里搜 "WaitCooldownChange" 实际上就是在搜这个函数。
```

```
UFUNCTION(BlueprintCallable,meta=(BlueprintInternalUseOnly = "true"))
```

```
static UWaitCooldownChange* WaitCooldownChange(UAbilitySystemComponent* AbilitySystemComponent,const FGameplayTag& InCooldownTag);
```

```
// 4. 结束任务
```

```
// 手动关闭这个节点, 清理垃圾
```

```
UFUNCTION(BlueprintCallable)
```

```
void EndTask();
```

```
protected:
```

```
// 5. 内部存储的数据
```

```
UPROPERTY()
```

```
TObjectPtr<UAbilitySystemComponent> ASC; // 记下我们要监听哪个组件
```

```
FGameplayTag CooldownTag; // 记下我们要监听哪个标签
```

```
// 6. 回调函数 (Callback)
```

// 这两个函数不是给蓝图调用的，而是给 ASC 调用的。

// 当 Tag 变了，ASC 调这个函数；当有 GE 加进来，ASC 调那个函数。

```
void CooldownTagChanged(const FGameplayTag InCooldownTag,int32  
NewCount);
```

```
void OnActiveEffectAdded(UAbilitySystemComponent*  
AbilitySystemComponent,const FGameplayEffectSpec&  
SpecApplied,FActiveGameplayEffectHandle ActiveEffectHandle);  
};
```

第三部分：源文件（.cpp）逐行代码解析

这个文件是“逻辑实现”。

1. 工厂函数：节点的诞生

这是你调用蓝图节点那一瞬间发生的事。

C++

// WaitCooldownChange.cpp

```
#include "AbilitySystem/AsyncTasks/WaitCooldownChange.h"
```

// 静态函数：创建并初始化任务节点

```
UWaitCooldownChange*  
UWaitCooldownChange::WaitCooldownChange(UAbilitySystemComponent*  
AbilitySystemComponent, const FGameplayTag& InCooldownTag)  
{
```

// [1] 创建一个新的对象实例。这就像是在内存里 new 了一个监听器。

```
UWaitCooldownChange* WaitCooldownChange =  
NewObject<UWaitCooldownChange>();
```

// [2] 把传进来的参数存起来，以后要用。

```
WaitCooldownChange->ASC = AbilitySystemComponent;
```

```
WaitCooldownChange->CooldownTag = InCooldownTag;
```

// [3] 安全检查：如果组件不存在，或者 Tag 无效，这个节点就废了，直接返回空。

```
if(!IsValid(AbilitySystemComponent)||!InCooldownTag.IsValid())
```

```
{
```

```
    WaitCooldownChange->EndTask();
```

```
    return nullptr;
```

```
}
```

// [4] 核心逻辑 A：监听“冷却结束”

// RegisterGameplayTagEvent：这是 ASC 自带的功能，意思是“当这个 Tag 的数量变化时，告诉我”。

// EGameplayTagEventType::NewOrRemoved：我们需要知道它什么时候“没了” (Removed)。

// AddUObject：绑定到我们自己的 CooldownTagChanged 函数上。

```
AbilitySystemComponent->RegisterGameplayTagEvent(
```

```
    InCooldownTag,
```

```
    EGameplayTagEventType::NewOrRemoved).AddUObject(
```

```
    WaitCooldownChange,
```

```
    &UWaitCooldownChange::CooldownTagChanged);
```

// [5] 核心逻辑 B：监听“冷却开始”

// OnActiveGameplayEffectAddedDelegateToSelf：当有任何 GE 应用到自己身上时，ASC 会广播这个委托。

// 我们绑定到 OnActiveEffectAdded，每当有 GE 加进来，我们就要去检查“是不

是冷却 GE? ”

```
AbilitySystemComponent->OnActiveGameplayEffectAddedDelegateToSelf.AddUObject(WaitCooldownChange,&UWaitCooldownChange::OnActiveEffectAdded);
```

```
// [6] 返回创建好的对象，交给蓝图系统去管理。
```

```
return WaitCooldownChange;
```

```
}
```

2. 结束任务：清理现场

当节点逻辑跑完，或者强制销毁时调用。如果不清理，游戏可能会崩溃（因为 ASC 还在往一个已经销毁的对象发消息）。

C++

```
void UWaitCooldownChange::EndTask()
```

```
{
```

```
// [1] 如果 ASC 还在（没被销毁）
```

```
if(!IsValid(ASC))return;
```

```
// [2] 取消监听 Tag 变化。如果不移除，ASC 还会一直发通知。
```

```
ASC->RegisterGameplayTagEvent(CooldownTag,EGameplayTagEventType::NewOrRemoved).RemoveAll(this);
```

```
// [3] 告诉引擎：“我这个任务做完了，可以把我当作垃圾回收（GC）掉了”。
```

```
SetReadyToDestroy();
```

```
MarkAsGarbage();
```

```
}
```

3. 响应函数 A：Tag 变了（检测冷却结束）

C++

```
void UWaitCooldownChange::CooldownTagChanged(const FGameplayTag InCooldownTag, int32 NewCount)
```

```

{
    // [1] NewCount 是当前这个 Tag 的层数。

    // 如果变成 0，说明身上没有这个 Tag 了 -> 冷却结束了！
    if(NewCount==0)
    {
        // [2] 触发蓝图的 CooldownEnd 引脚，传 0.0 秒过去。
        CooldownEnd.Broadcast(0.f);
    }
}

```

4. 响应函数 B：有新 GE 来了（检测冷却开始）

这个函数稍微复杂一点，因为它要过滤出“这是不是我要的冷却 GE”。

C++

```

void UWaitCooldownChange::OnActiveEffectAdded(UAbilitySystemComponent*
AbilitySystemComponent, const FGameplayEffectSpec& SpecApplied,
FActiveGameplayEffectHandle ActiveEffectHandle)
{
    // [1] 准备容器，用来装这个新 GE 携带的所有 Tag
    FGameplayTagContainer AssetTags;

    // [2] 获取 GE 本身的 Tag (AssetTags)
    SpecApplied.GetAllAssetTags(AssetTags);

    FGameplayTagContainer GrantedTags;

    // [注意]：你这里的代码有一点点小瑕疵（不影响运行但逻辑有点怪）：
    // 你把 AssetTags 传给了 GetAllGrantedTags，这意味着 Grant 的 Tag 也被塞
    进了 AssetTags 变量里。

    // 下面的 GrantedTags 变量其实是空的。不过没关系，只要 AssetTags 里有就
    行。

```

```

SpecApplied.GetAllGrantedTags(AssetTags);

// [3] 检查：这个 GE 是否带有我们要监听的 CooldownTag?

// 比如：新来的 GE 是“火球术冷却”，带有 Ability.Fire.Cooldown 标签。

// 而我们正在监听 Ability.Fire.Cooldown。匹配成功！

if(AssetTags.HasTagExact(CooldownTag) ||
GrantedTags.HasTagExact(CooldownTag))

{

    // [4] 我们需要查询这个 Tag 还要持续多久。

    // FGameplayEffectQuery 是一个查询过滤器，我们告诉它：“只查带有
CooldownTag 的效果”。

    const FGameplayEffectQuery GameplayEffectQuery =
FGameplayEffectQuery::MakeQuery_MatchAnyOwningTags(CooldownTag.GetSingleTag
Container());

    // [5] 准备一个数组来存查到的时间（因为可能身上有好几个同名 Tag）

    TArray<float> TimesRemaining =
AbilitySystemComponent->GetActiveEffectsTimeRemaining(GameplayEffectQuery);

    // [6] 如果查到了时间

    if(TimesRemaining.Num() > 0)

    {

        // [7] 取出最大的那个时间（通常冷却只有一个，但如果有多，取最长
的那个）

        float TimeRemaining = TimesRemaining[0];

        for(int32 i=0;i<TimesRemaining.Num();i++)

        {

            if(TimesRemaining[i] > TimeRemaining)

```



```

        {
            TimeRemaining = TimesRemaining[i];
        }
    }

    // [8] 触发蓝图的 CooldownStart 引脚, 告诉 UI: “冷却开始! 还剩 X
秒”。

    CooldownStart.Broadcast(TimeRemaining);
}
}
}
}

```

总结：数据是怎么流动的？

1. **UI 蓝图** 调用 WaitCooldownChange 节点，传入 Ability.Fire 标签。
2. **C++** 创建了一个监听器，悄悄趴在 ASC 上。
3. **玩家释放火球术** -> 一个名为 GE_Fire_Cooldown 的效果应用到玩家身上。
4. **ASC** 发现有 GE 应用，通知监听器 (OnActiveEffectAdded)。
5. **监听器** 检查：这 GE 有 Ability.Fire 标签吗？有！
6. **监听器** 算出剩余时间（比如 1.5 秒），通过 CooldownStart 告诉 UI。
7. **UI** 收到 1.5 秒，开始转圈圈倒计时。
8. **时间流逝**... 1.5 秒过去了。
9. **ASC** 发现 Tag 数量归零了，通知监听器 (CooldownTagChanged)。
10. **监听器** 发现 Count=0，通过 CooldownEnd 告诉 UI。
11. **UI** 收到结束信号，隐藏转圈圈，点亮图标。

这就是这个冷却节点设计的全貌！